

A document-intelligence RAG platform, traced from an uploaded PDF to a cited, streamed answer. Every stack choice below, and the one question that actually matters — why a fusion algorithm sits on top of pgvector when pgvector already does semantic search — is pulled directly from the two runtimes in the repo: the Next.js app and the Python retrieval backend.

In-repo name: **Cortex**

Next.js **16** · React **19**

Two runtimes: **Next.js proxy** + **FastAPI retrieval engine**

Every model call: **Gemini free tier**, one API key

Overview

Full tech stack

Data model

Auth & security

Upload → storage

Structural parsing

C

01 What Cortex is

Cortex lets a user upload documents, or connect a Google Drive folder, and then ask questions against them conversationally. The product bet is narrow and specific: never answer from the model's own knowledge, only from retrieved passages, and always show which passage — down to the page number — backed the claim. Everything downstream of that bet (hybrid search, reranking, a self-grading step before the model is allowed to speak) exists to make the retrieval half of that promise actually hold up.

8

Postgres tables: workspaces, documents, document_chunks, chat_sessions/messages, connector_accounts, drive_sync_state

2

retrieval RPCs in the schema — one workspace-scoped, one ACL-aware — both doing dense + lexical + RRF

6-stage

query pipeline: embed → hybrid retrieve → rerank → grade → (rewrite) → stream

768-dim

every embedding, truncated from gemini-embedding-001's native size via Matryoshka slicing

A STRUCTURAL NOTE

The Next.js app and the Python backend are two separate deployable services that share one Supabase project. The frontend never talks to Postgres for anything retrieval-related except one legacy route ([see §05](#)) – every chat turn is a proxied call to `POST /v1/query` on the FastAPI service.

02 The full stack

Grouped by the job each piece is actually doing, not the marketing category.

Frontend & UI — cortex/src

PIECE	WHAT IT'S DOING HERE
<code>Next.js 16</code> App Router	Server-rendered dashboard, auth pages, and a thin API layer (<code>src/app/api/*</code>) that proxies chat/upload/connector calls to the Python backend, re-attaches human-readable metadata to bare chunk ids, and owns everything session-shaped: message persistence, session titles, conversation history.
<code>React 19 + TypeScript</code>	Standard app shell; <code>src/types.d.ts</code> and Zod-free hand-typed API contracts keep the two runtimes' JSON in sync manually rather than via a shared schema package.
<code>Tailwind v4 + Framer Motion / GSAP</code>	Landing page and dashboard animation – scroll-aware navbar, the animated architecture pipeline on the marketing page, Lenis-driven smooth scroll.
<code>react-markdown + remark-gfm</code>	Renders streamed assistant answers (which arrive as raw Markdown with inline <code>[chunk_id]</code> citation tokens) into formatted chat bubbles.
<code>mammoth, pdf2json, papaparse</code>	Client-side preview/parsing helpers for the document reader panel – separate from the server-side Docling pipeline that actually produces the retrieval index.
<code>Radix UI primitives, lucide-react</code>	Accessible dialog/label primitives and one consistent icon set.

AI & retrieval — cortex/backend

PIECE	WHAT IT'S DOING HERE
<code>gemini-embedding-001</code>	One embedding model for every chunk and every query, truncated to 768 dims on both the ingest and query side so the two are always byte-comparable. Calling the exact same model+slice from both the Next.js quick-search route and the Python backend is what keeps the two retrieval paths compatible with the same <code>vector(768)</code> column.
<code>gemini-3.1-flash-lite</code>	A cheap, structured-output model reused for three unrelated jobs: reranking (cross-encoder substitute), CRAG relevance grading, and document auto-summarization on ingest. All three ask for JSON via <code>responseSchema</code> , never free text.
<code>gemini-2.5-flash</code>	The answer-writing model. Streamed manually over <code>:streamGenerateContent?alt=sse</code> with hand-rolled SSE line parsing and retry-on-5xx, rather than through Google's SDK, so the backend controls exactly when a partial stream is allowed to fail.
<code>Docling</code>	IBM's layout-aware document converter. Does the real work pdf-parse-style libraries can't: heading hierarchy, table → Markdown reconstruction, and a per-page OCR-confidence grade, for PDF/DOCX/XLSX specifically.
<code>FastAPI + asyncpg + uvicorn</code>	The retrieval engine. Async end to end so one query request can run the dense and lexical branches of hybrid search concurrently (<code>asyncio.gather</code>) instead of sequentially.
<code>python-jose</code>	Verifies the Supabase-issued bearer token against the project's own JWKS (ES256/RS256) — the Python service trusts nothing from the Next.js layer, only a token it independently validated.

Data & infra

PIECE	WHAT IT'S DOING HERE
<code>Supabase Postgres</code>	Auth, Storage, and every relational table in one instance — Row Level Security scopes every query to the caller by default, so "check ownership" is a database-enforced property, not just an application-code habit.
<code>pqvector</code>	The dense half of retrieval. indexed with an IVFFlat list

<code>ivfflat, cosine</code>	index over <code>vector_cosine_ops</code> . Lives in the same table as the FTS column and the ACL JSON — one join gets you a filtered, ranked, permission-safe result.
<code>Postgres full-text search</code> <code>GIN + tsvector</code>	The lexical/BM25-style half — <code>ts_rank_cd</code> over <code>websearch_to_tsquery</code> , GIN-indexed on <code>to_tsvector('english', content)</code> .
<code>Supabase Vault</code>	Stores the Google Drive OAuth refresh token. The <code>connector_accounts</code> table only ever holds a <code>refresh_token_secret_id</code> — a pointer, never the secret.
<code>Upstash Redis + @upstash/ratelimit</code>	A 20-req/min sliding-window limiter on chat, keyed by user id, that fails <i>open</i> (logged to Sentry) rather than blocking the product on a Redis outage.
<code>Sentry + PostHog</code>	Error capture on every stream branch of the chat/upload proxies (rate-limit, persistence, follow-ups all wrapped individually), and product analytics on the Next.js side.

03 **Data model**

One Supabase project holds both the relational app data and the vector index — there is no separate vector database to keep in sync.

TABLE	HOLDS	NOTABLE COLUMNS
<code>workspaces / workspace_members</code>	Tenant boundary. One owner, optional admin/member roster.	<code>owner_id</code> , role check <code>admin member</code>
<code>documents</code>	One row per uploaded or Drive-synced file.	<code>storage_path</code> , <code>summary</code> , <code>topics jsonb</code> , <code>external_id</code> (Drive file id), <code>source_type</code>
<code>document_chunks</code>	The retrieval index itself — every chunk of every	<code>embedding vector(768)</code> , <code>embedding_model</code> (guards against silently mixing incompatible vectors on a model

	document.	swap), acL_permissions jsonb, metadata jsonb (page_number, headers[], is_table)
chat_sessions / chat_messages	Conversation history.	sources jsonb, answered_from (documents none) – lets the UI label whether an answer was actually grounded
connector_accounts / drive_sync_state	One OAuth connection per (user, provider); per-folder delta-sync cursor.	refresh_token_secret_id (Vault pointer, not the token), last_synced_at

READ FROM THE SCHEMA'S OWN COMMENTS

`schema.sql` documents its own hardening pass inline – several UPDATE/DELETE RLS policies (on `documents`, `document_chunks`, `workspace_members`) are annotated as having been *missing* in an earlier version, silently no-op'ing features like renaming a workspace or persisting an auto-summary. Worth knowing this schema has already been through one real security-hardening cycle, not just a first draft.

04 Auth & security

Two independent, overlapping layers of access control – deliberately redundant, because they guard different attack surfaces.

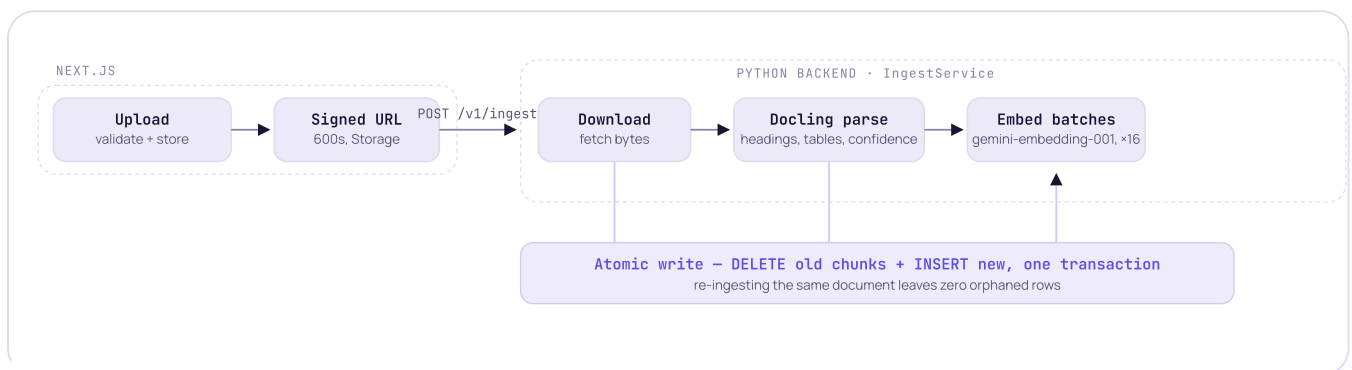
- **Postgres RLS** on every table, scoped through the ownership chain (`workspace` → `document` → `chunk`) via `auth.uid()`. This is what stops a direct table read from leaking across tenants.
- **An explicit ACL check inside the retrieval RPC itself** – `acL_permissions` is checked in a `visible_chunks` CTE *before* either ranked list is built, because Drive-connector sharing is finer-grained than plain workspace ownership (a chunk can be marked `{'{'}"public": true{'}'}` or shared to specific user ids). RLS alone can't express that; the RPC's own WHERE clause does.
- **JWT verification independent of the frontend** – the Python backend fetches the Supabase project's JWKS itself (`/auth/v1/.well-known/jwks.json`, cached 10 min, ES256/RS256) and verifies every bearer token from scratch. A compromised Next.js layer

can't forge a backend-trusted identity without a token Supabase actually issued.

- **Storage bucket policies are folder-scoped**, not just bucket-scoped — `(storage.foldername(name))[1]` must match a workspace the caller owns, since the object path convention is `<workspaceId>/<timestamp>_<filename>`.

05 Ingest: upload to indexed chunk

A file crosses from the browser to a queryable vector row through six handoffs, split across both runtimes:



A re-upload of the same document never accumulates duplicate chunks — the delete-then-insert runs inside a single asyncpg transaction.

06 Structural parsing, not text extraction

`services/parser.py` wraps Docling rather than a plain PDF-to-text library specifically because retrieval quality depends on more than raw words:

- **Heading breadcrumbs are baked into the chunk text** — every chunk is prefixed with `[Context: Section A > Subsection B]` before it's embedded, so a bullet point retrieved in isolation still carries which part of the document it came from, both for the embedding model and for the human reading the citation.
- **Tables are their own chunk type**, exported as clean Markdown rather than left as visually-broken running text — a table read as prose loses its row/column relationships entirely.
- **Every chunk carries a confidence score** derived from Docling's per-page OCR quality grade (excellent/good/fair/poor → 0.95–0.30), discounted further for ungridded tables. This isn't surfaced to the end user yet, but it's already on every row for when it is.
- **Small adjacent chunks under the same heading are merged** up to a max size, so a one-line paragraph doesn't become its own low-context vector.

07 Chunking & embeddings

`services/embeddings.py` calls `gemini-embedding-001` and keeps only the first 768 values of the returned vector — Matryoshka Representation Learning means the model was trained so its embedding stays meaningful when truncated, not just zero-padded. That single design choice does two things at once:

~75%

smaller index + storage footprint than the model's native embedding size, at a fraction of the quality cost

1

byte-identical slice used on *both* sides — the Python ingest path and the Next.js quick-search route call the same model with the same truncation, so their vectors are directly comparable in the one `vector(768)` column

Every chunk row also stamps `embedding_model` at write time — not because the model changes today, but so that a future swap can't silently mix two incompatible vector spaces in the same similarity search without it being detectable.

08 Two retrieval paths, one schema

There isn't one query pipeline in this codebase — there are two, at different points in the product's life, both still live.

LEGACY • QUICK SEARCH

`src/app/api/search/route.ts` embeds the query with the JS Gemini SDK directly from Next.js and calls `match_documents()` — workspace-scoped only, no per-chunk ACL, no Python backend involved at all.

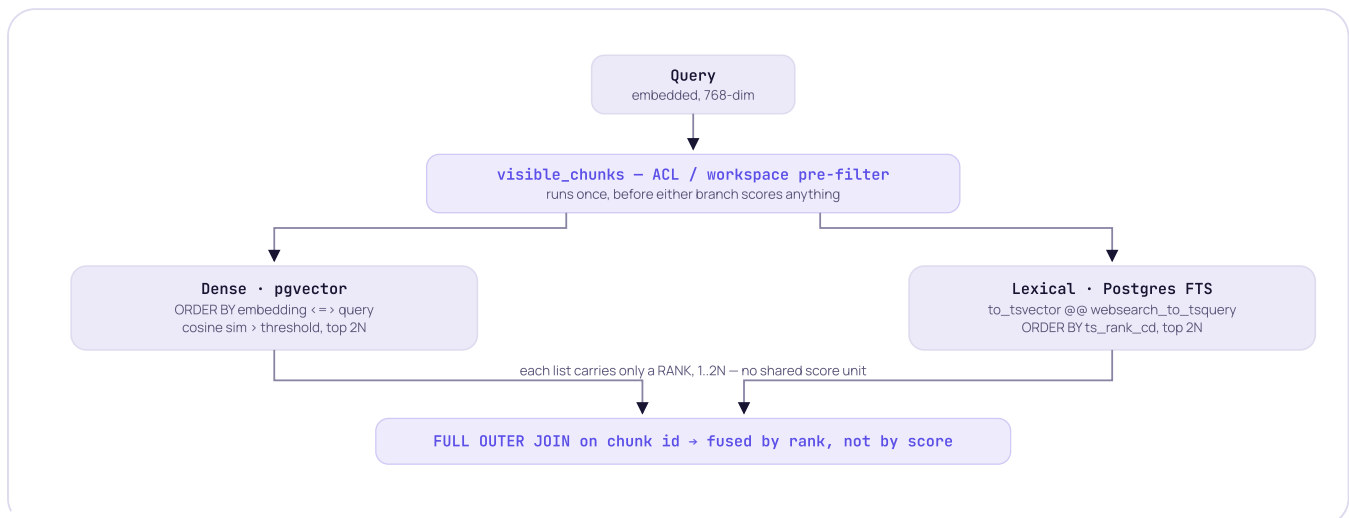
CURRENT • CHAT / RAG

`POST /v1/query` on the FastAPI backend calls `match_hybrid_documents()` — the same dense+lexical+RRF math, wrapped in an ACL pre-filter, followed by reranking, CRAG grading, and streamed synthesis.

Both RPCs run the identical fusion algorithm because the second was built as a superset of the first, not a replacement — `match_documents()` is explicitly left in place in the migration comments ("current app depends on it"). See §16 for what that split means in practice.

09 Hybrid search: two retrievers, run in parallel

Both RPCs (and the Python-side `HybridRetriever._retrieve_app`) do the same three things: filter to what the caller may see, run a dense pass and a lexical pass over that filtered set, then fuse the two ranked lists.



Both branches see the same ACL-filtered rows and are capped at 2× the requested count before fusion – the join is small, not a re-scan of the table.

10 Why RRF, when pgvector already does dense search

This is the question worth actually answering carefully, because the reflex answer – "hybrid search is better" – skips the part that's specific to *this* codebase: RRF was never a replacement for pgvector. Dense search stayed exactly as it was; RRF is the layer that let a second, structurally different retriever sit next to it without breaking anything.

Dense and lexical search fail on opposite query shapes

Cosine similarity over `gemini-embedding-001` vectors is strong on paraphrase and intent – "how do I stop my subscription" finds a passage titled "cancellation policy" with no shared words at all. It is weak on the exact tokens embeddings are built to abstract away: acronyms, part numbers, error codes, a phrase copy-pasted verbatim from the source document. Postgres FTS is the mirror image – excellent at exact/keyword recall, blind to paraphrase. Keeping only one retriever means every query type the other one was good at silently stops working.

The two retrievers don't even share a score unit

Dense similarity is a cosine distance mapped into $[0, 1]$. `ts_rank_cd` is an unbounded, corpus-dependent term-frequency score with no natural ceiling. Averaging or weighting these two numbers directly ($0.5 \times \text{dense} + 0.5 \times \text{lexical}$) requires normalizing one or both first, and picking a weight that will need re-tuning every time the corpus or query mix shifts.

```
-- backend/db/migrations/0001_..._hybrid_rpc.sql - the fusion
itself SELECT COALESCE(v.id, b.id) AS id, COALESCE(1.0 / (60 +
v.rank), 0.0) + COALESCE(1.0 / (60 + b.rank), 0.0) AS rrf_score
FROM vector_search v FULL OUTER JOIN bm25_search b ON v.id = b.id
ORDER BY rrf_score DESC LIMIT match_count;
```

RRF only ever looks at rank — each chunk's position (1st, 2nd, 3rd...) within a list — never the raw score. That makes it scale-free: it doesn't matter that one list's numbers live in $[0, 1]$ and the other's don't, because neither number is ever read. A chunk that ranks highly in *either* list pulls its combined score up; a chunk both retrievers agree on compounds. One constant, $k=60$, replaces what would otherwise be a hand-tuned weighted-sum formula.

Where the exact same math runs twice

- `schema.sql → match_documents()` — the original, workspace-only RPC, still backing the legacy quick-search route.
- `migrations/0001 → match_hybrid_documents()` — the same dense+BM25→RRF fusion, now behind an ACL pre-filter, used by `RETRIEVAL_MODE=rpc` (the default).
- `services/retrieval.py → reciprocal_rank_fusion()` — a line-for-line Python port of the identical algorithm, used when `RETRIEVAL_MODE=app` runs dense and lexical as two concurrent `asyncio` queries instead of one SQL statement — kept because it's mockable in the integration test suite (`tests/_fakes.py`) in a way a black-box RPC call isn't.

THE HONEST ONE-LINE ANSWER

pgvector only ever gave the system one retrieval signal — semantic similarity. RRF is what let a second, independent signal (exact lexical match) join the ranking without requiring the two signals to agree on what a "score" even means. It's not competing with pgvector; pgvector is one of its two inputs.

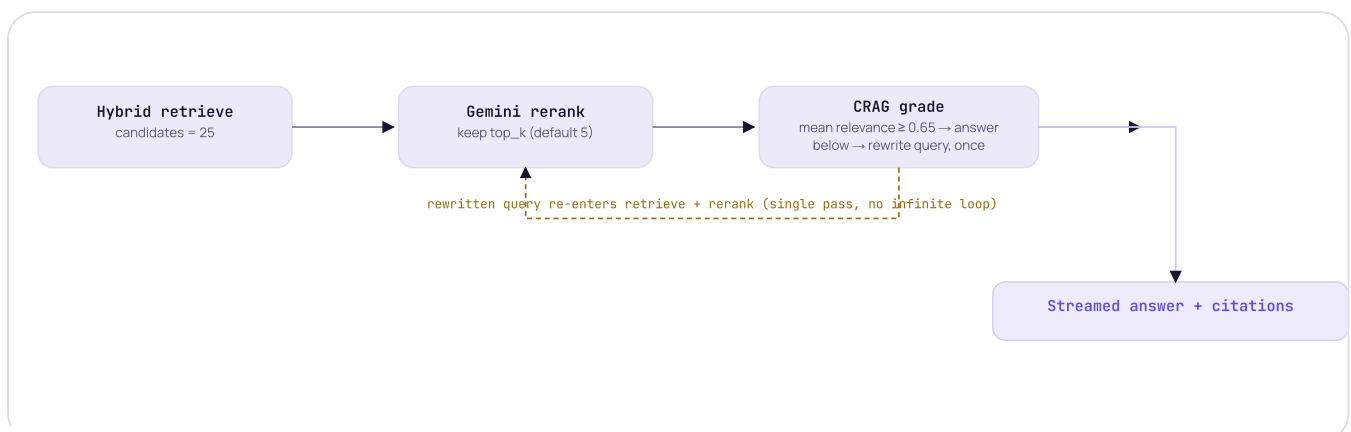
11 Reranking: Gemini as a cross-encoder stand-in

`services/retrieval.py` + reranker) sends an `Embedder` 20 passed candidates to Gemini's `gemini-2.5-flash-lite` in one call, asking for a 0–1 relevance score per passage via a strict JSON schema — a cheap approximation of a real cross-encoder, chosen because Cohere's Rerank API has no usable free production tier.

The failure mode is deliberately boring: if the model call errors, returns malformed JSON, or scores nothing valid, the reranker logs a warning and falls back to the RRF fusion order truncated to `top_n` — a chat turn never fails just because the reranking step did.

12 Corrective RAG: a grade before the model is allowed to speak

Between retrieval and synthesis sits one more model call: a grader that decides whether the retrieved context is actually good enough to answer from.



Every stage degrades gracefully on failure — a broken reranker call falls back to fusion order, a broken grader call falls back to "answer anyway" — so a model hiccup never surfaces as a hard error to the user.

The rewrite path only ever fires once per turn — there's no recursive loop chasing a perfect context set, which bounds worst-case latency to two retrieval rounds instead of an open-ended search.

13 Synthesis: citations the model can't fabricate

`services/synthesis.py` streams `gemini-2.5-flash` with a system prompt that is one long constraint: answer only from the numbered context blocks, cite every factual claim inline as `[chunk_id]`, never invent an id, say plainly when the context doesn't contain the answer.

The trick that actually enforces this: the citation list shown to the user is never generated

the check that actually enforces this: the citation list shown to the user is never generated by the model. It's built by the orchestrator from the same ranked-chunk objects handed to the model as context, and emitted as its own citations SSE event before the answer even starts streaming. The model's inline [id] tokens are a display aid inside the answer text; cited_ids() exists to cross-check them against that authoritative list, not the other way around.

14 Google Drive connector

`integrations/gdrive.py` — OAuth connect, a per-folder delta sync keyed by Drive's `modifiedTime`, and an optional background re-sync loop (`SYNC_INTERVAL_MINUTES`, 0 = on-demand only).

- The OAuth state parameter is HMAC-signed (`CONNECTOR_STATE_SECRET`) so the callback can't be replayed or forged with a different user id.
 - A changed file is replaced atomically — same delete-then-insert transaction pattern as the upload path — keyed by `external_id` rather than `document_id`, since Drive-sourced chunks are looked up by Drive's file id during sync.
 - Per-folder sync state (`last_synced_at`, `last_status`, `last_report`) lets the dashboard show sync health without re-running a sync just to check it.
-

15 Rate limits, retries & observability

- **Chat rate limiting** — 20 requests/minute per user (Upstash sliding window), fails open on a Redis error rather than blocking the product, with the failure itself reported to Sentry.
 - **Every outbound Gemini call retries on 429/500/502/503/504** with exponential backoff (`core/retry.py`), and the synthesis stream specifically refuses to retry once any token has already reached the client — a partial answer is never silently restarted from zero.
 - **Sentry wraps each stream branch individually** in the chat and upload proxies (`stage: "ratelimit" | "persist_assistant" | "follow_ups" | "chat_proxy"`) so a failure in a non-critical side effect (like generating follow-up questions) is visible without taking down the whole response.
-

16 Flagged: gaps between the pitch and the code

Worth knowing before this description goes in front of anyone else — none of these are bugs exactly, but they're seams a careful reader will find.

MARKETING VS. IMPLEMENTATION

The landing page and root README describe an "Agentic Decision" step that reaches out to **live web search via Tavily** when document context is weak. There is no Tavily integration anywhere in the code — the only matches are the marketing copy itself